

מבני נתונים

תרגיל 1 – חישוב זמן ריצה של תכניות מחשב

הגדרות:

זמן ריצה היא פונקציה.

הקלט לפונקצית זמן הריצה: גודל הקלט (כמות הנתונים הבסיסיים) לקטע הקוד.

הפלט של פונקציית זמן הריצה: כמות הפעולות הבסיסיות שמבצע קטע הקוד.

פעולה בסיסית

היחידה האריתמטית-לוגית מסוגלת לבצע שלושה סוגים של פעולות בסיסיות (הרחבה תלמד במסגרת הקורס מבנה המחשב ותורת המיתוג):

- פעולות אריתמטיות
למשל, חיבור או חיסור שני מספרים. אוסף הפעולות האריתמטיות שיכולה היחידה לבצע עלול להיות דל ביותר עד כדי מצב שבו בחלק מן הדגמים לא מומשה ישירות האפשרות של כפל וחילוק (במקום מימוש ישיר בחומרה, יש להשתמש בתוכנות המבצעות כפל וחילוק באמצעות אלגוריתם).

- פעולות על מחרוזות בזיכרון המחשב
- פעולות לוגיות
פעולות השוואה בעיקר. למשל, בהינתן שני מספרים, קבע האם הם שווים, ואם לא, מי מהם גדול יותר.

פסאודו קוד: תיאור מצומצם ולא רשמי לאלגוריתם של תכנית מחשב.

סדרה חשבונית:

יהי $a_1, a_2, \dots, a_n$ סדרה של מספרים.
הסדרה תקרא סדרה חשבונית אם לכל $n \geq 2$ מתקיים $a_n = a_{n-1} + d$ עבור d כלשהו.
נסמן ב- s_n את סכום n האיברים הראשונים של הסדרה.
עבור סדרה חשבונית מתקיים: $a_n = a_1 + (n-1) \cdot d$ וגם $s_n = \frac{n(a_1 + a_n)}{2}$.

סדרה הנדסית:

יהי $a_1, a_2, \dots, a_n$ סדרה של מספרים כך שלכל n מתקיים $a_n \neq 0$.
הסדרה תקרא סדרה הנדסית אם לכל $n \geq 2$ מתקיים $a_n = a_{n-1} \times q$ עבור q כלשהו.
נסמן ב- s_n את סכום n האיברים הראשונים של הסדרה.
עבור סדרה הנדסית מתקיים: $a_n = a_1 \times q^{n-1}$ וגם $s_n = \frac{a_1 \times (q^n - 1)}{q - 1}$.

שאלות:

1. חשבו את פונקציית זמן הריצה המדויקת של התכניות הבאות כקלט של n (בניח לצורך התרגיל כי Printf() היא פעולה בסיסית):

א.

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d,", A[i]);
}
```

ב.

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i+=2)
        printf("%d,", A[i]);
}
```



מכון טכנולוגי חולון
Holon Institute of Technology

.ג

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n/2; j++) {
            k += 1;
            A[i] = k
        }
}
```

.ד

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}
```

```
void fun(int *A, int n)
{
    int i, j, k0 = 0, k1 = 1;
    for (i = 0; i < n; i++)
        for (j = 0; j < i; j++)
        {
            k0 += 1;
            k1 *= 2;
            A[i]=k0+k1
        }
}
```

2. חשבו את זמן הריצה של התכניות הבאות:

א.

```
void fun(int *A, int n)
{
    int i = 0;
    while (i < n) {
        A[i] += 1;
        i++;
    }
}
```

ב.

```
void fun(int *A, int n)
{
    int i = 0;
    while (i < n) {
        A[i] += 1;
        i += 3;
    }
}
```

ג.

```
void fun(int *A, int n)
{
    int i = 1;
    while (i < n) {
        A[i]++;
        i *= 2;
    }
}
```

ד.

```
void fun(int *A, int n)
{
    int i = 2;
    while (i < n) {
        A[i] = A[i - 1]; // כל השורה הנ"ל מהווה פעולה בסיסית אחת
        i = i * i;
    }
}
```

3. חשבו את זמן הריצה של התכניות הבאות:

א.

```
void fun(int *A, int n)
{
    int k;
    k = n;
    while (k > 1) {
        k = k/5;
        for (j = 1; j <= log(n); j++)
            for (i = 1; i <= n/2; i++)
                A[i] += 1;
    }
}
```

ב.

```
void fun(int *A, int n)
{
    int k = 2, m = 0;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++)
            m += 1;
        while (k <= i)
            k = k * k;
    }
    A[m%n] += k; // כל השורה הנ"ל מהווה פעולה בסיסית אחת
}
```

4. חישובו את זמן הריצה של אלגוריתם Bubble Sort:

```
BubbleSort(A, n) (A – array of integers, n – A's size)
  swapped = True
  while swapped = True do:
    swapped = False
    n = n - 1
    for i = 1 to n do:
      if (A[i] > A[i + 1])
        then swap(A[i], A[i + 1])
        swapped = True
    end for
  end while
```

5. חישובו את זמן הריצה של אלגוריתם Selection Sort:

```
SelectionSort(A, n) (A – array of integers, n – A's size)
  for i = 1 to (n - 1) do :
    smallest=i
    for j =(i+1) to n do :
      if (A[j] < A[smallest])
        then smallest=j
    end for
    swap(A[i], A[smallest])
  end for
```